

Device Access Control with AAA – Authentication, Authorization and Accounting



ICT2203 Networks Security

LAB 8 NOTES

2021-2022 Trimester 1

LEARNING OUTCOMES

In this lab you will learn ...

- **Authentication, Authorization and Accounting (AAA)**
 - a common security framework for mediating network and application access
- Management plane can be accessed via:
 - Console
 - Telnet
 - SSH
- Security of management plane is commonly achieved via Authentication, Authorization and Accounting (AAA).



Authentication - Who is allowed access?



Authorization - What are they allowed to do?



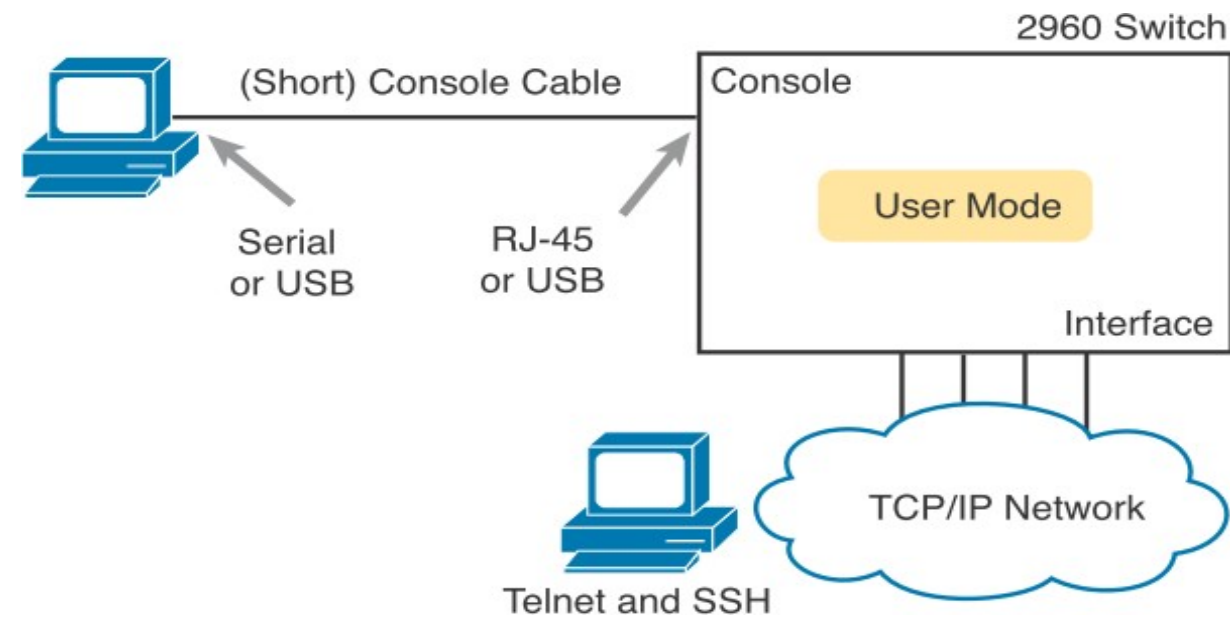
Accounting - What did they do?

Securing the Management Plane (Cisco NFP Framework)



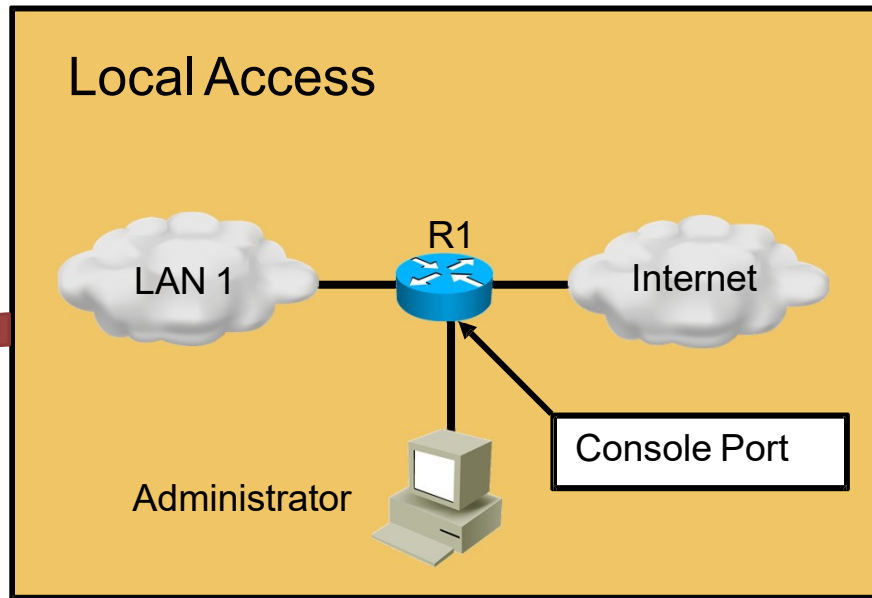
Under Cisco NFP framework, another component to secure is the **management plane**, which is a collection of protocols and access methods to *configure, manage, and maintain* network devices.

In fact, you have been accessing the **management plane** since ICT1010 when you connect your computer using the **console** cable to the network devices to configure them.

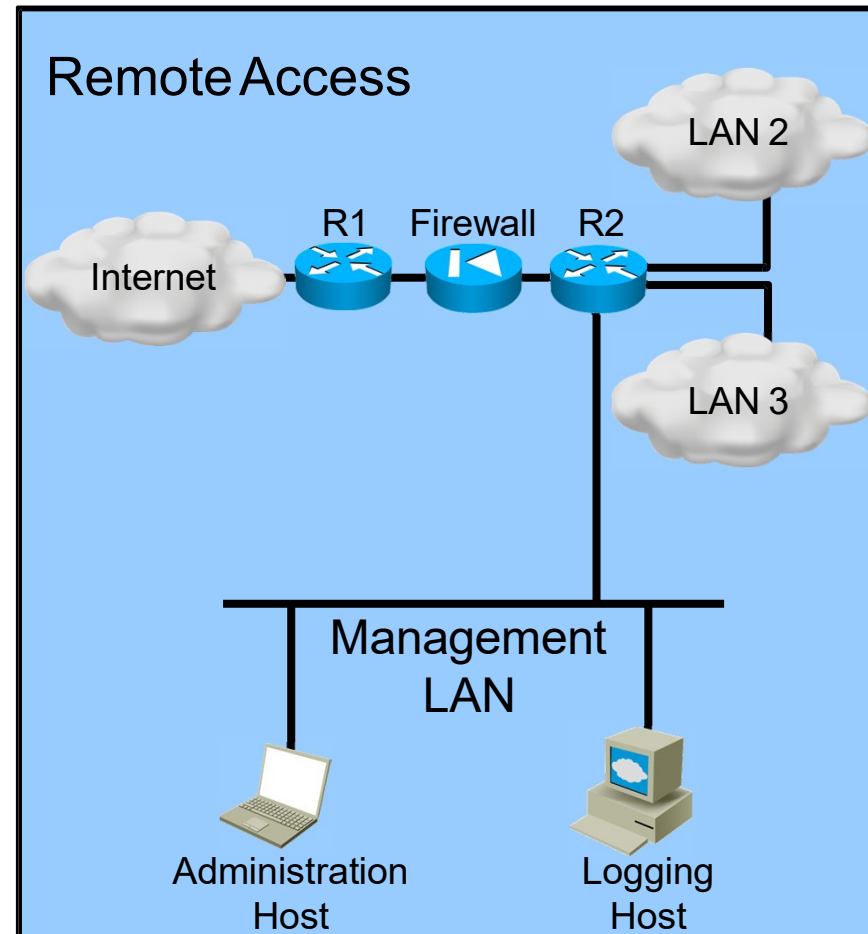


Besides console cable, you can also remotely access the **management plane** via **Telnet** or **SSH**.

Local access via console is the most secure. But for convenience, remote access via Telnet or SSH is commonly implemented in practice.



Requires a direct cable connection from the administrator host to the console port of the network device.

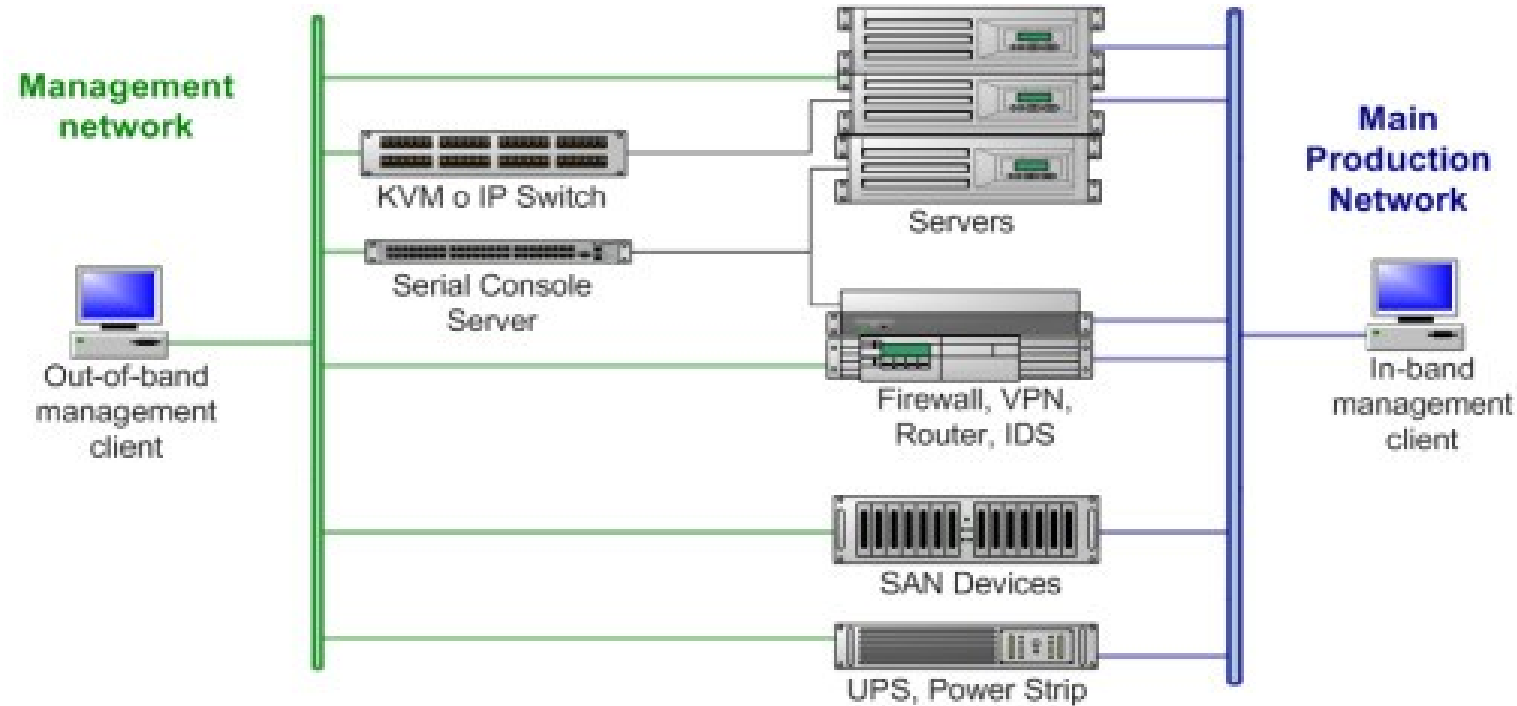


Uses Telnet or SSH connections over the LAN from the administrator host to the network device.

To be secure in remote management, **out-of-band (OOB) network** may be constructed by using isolated **management port**, or **virtual routing and forwarding (VRF)** technology.

Notice that now every device has an additional link to connect to the **OOB management network**

Although the devices seem to link the **management network** and main **production data network** together, traffic are not able to flow between them due to the use of **isolated management port** and **VRF**.



Management port is a layer 3 port specially constructed for device management which is isolated from the normal ports.

Configuring **management port** on Catalyst 2960-X layer-2 switch:

```
S1(config)# int fastethernet0
S1(config-if)# ip address ...
S1(config-if)# no shut
S1(config)# ip default-gateway
```

...

Similarly for Catalyst 3650
layer-3 switch:

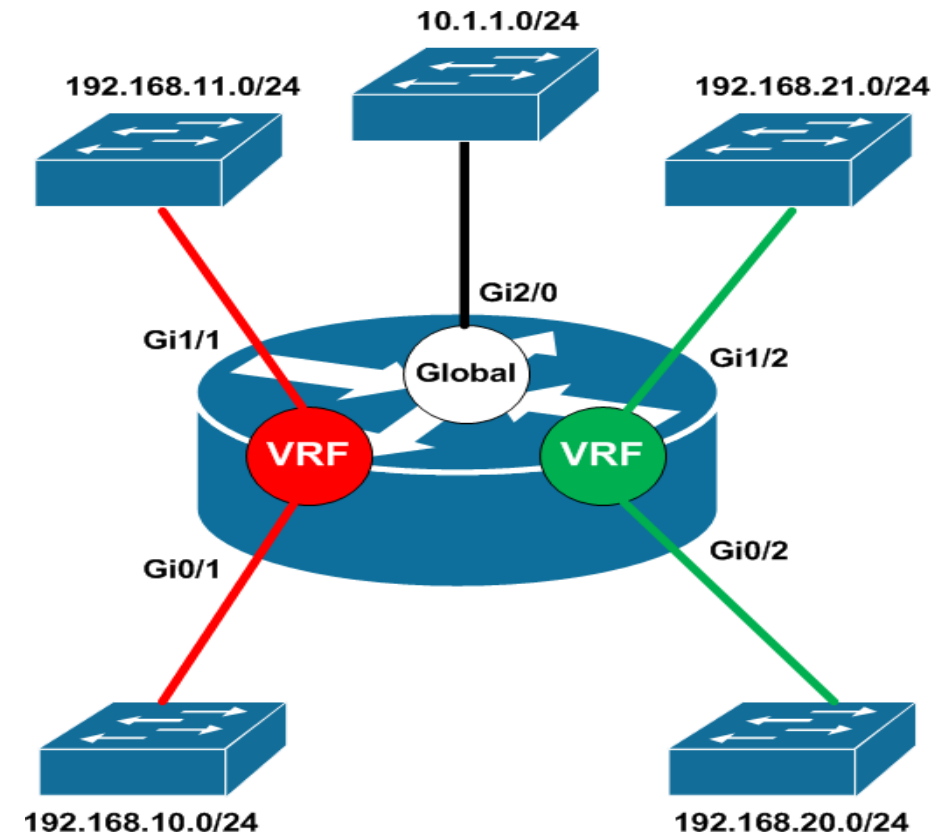
```
S1(config)# int gigabitethernet0/0
S1(config-if)# ip address ...
S1(config-if)# no shut
S1(config)# ip default-gateway ...
```



If management port is not available, **Virtual Routing and Forwarding (VRF)** technology may be used for management LAN, e.g. in Cisco routers

A router with **VRF** will support **multiple instances** of **routing tables**, essentially dividing a physical router into multiple logical routers:

- A **global routing table** which exists by default
- **Separate routing tables** for different VRF instances, e.g. **Red** and **Green** VRF as shown
- Each interface will either belong to global or a VRF instance
- Traffic on different VRFs are separated because of different routing tables



For Cisco ISR 2900 routers, some of the commands for **VRF** are as follows:

Create a VRF instance, e.g. named Mgmt-vrf:

```
Router(config) # vrf definition Mgmt-vrf
```

Associate an interface to VRF, e.g. g0/0:

```
Router(config) # int g0/0  
Router(config-if) # vrf forwarding Mgmt-vrf
```

Configure static route into specific VRF routing table:

```
Router(config) # ip route vrf Mgmt-vrf ...
```

Ping through a specific VRF:

```
Router# ping vrf Mgmt-vrf ...
```


To verify the **routing table** of **VRF**, use the following command:

```
Router# show ip route vrf Red
Codes: C - connected, S - static, R - RIP, M - mobile, B - BGP
       D - EIGRP, EX - EIGRP external, O - OSPF, IA - OSPF inter area
       N1 - OSPF NSSA external type 1, N2 - OSPF NSSA external type 2
       E1 - OSPF external type 1, E2 - OSPF external type 2, E - EGP
       i - IS-IS, su - IS-IS summary, L1 - IS-IS level-1, L2 - IS-IS level-2
       ia - IS-IS inter area, * - candidate default, U - per-user static route
       o - ODR, P - periodic downloaded static route

Gateway of last resort is not set

C      192.168.10.0/24 is directly connected, GigabitEthernet0/1
C      192.168.11.0/24 is directly connected, GigabitEthernet1/1
```

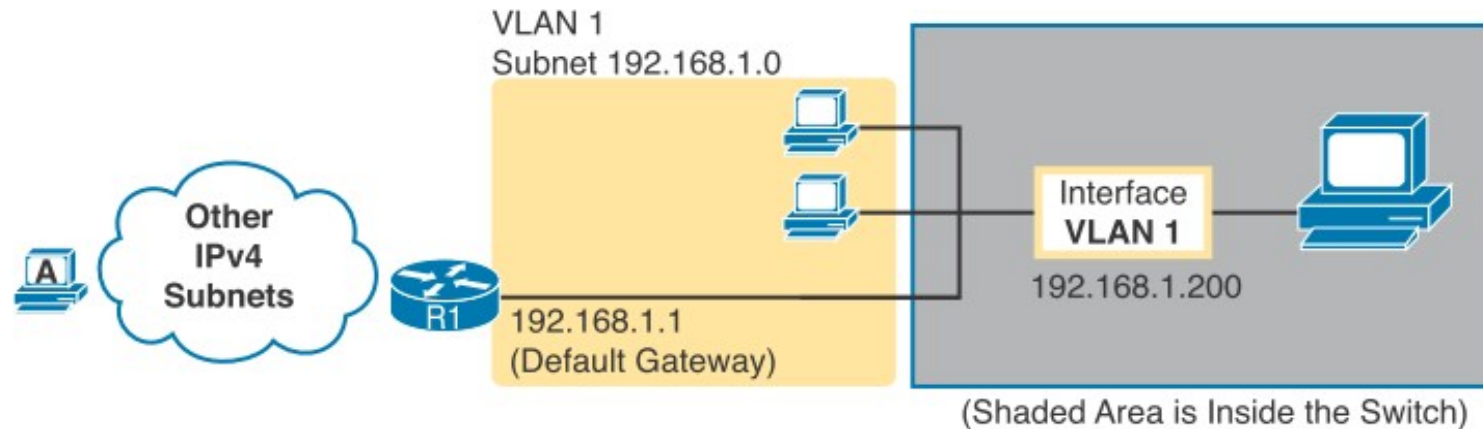
For comparison:

```
Router# show ip route
Codes: C - connected, S - static, R - RIP, M - mobile, B - BGP
       D - EIGRP, EX - EIGRP external, O - OSPF, IA - OSPF inter area
       N1 - OSPF NSSA external type 1, N2 - OSPF NSSA external type 2
       E1 - OSPF external type 1, E2 - OSPF external type 2, E - EGP
       i - IS-IS, su - IS-IS summary, L1 - IS-IS level-1, L2 - IS-IS level-2
       ia - IS-IS inter area, * - candidate default, U - per-user static route
       o - ODR, P - periodic downloaded static route

Gateway of last resort is not set

C      10.1.1.0/24 is directly connected, GigabitEthernet2/0
```

For remote management, network devices will also need **IP addresses**. Thus, a switch will need to be configured with IP address using **switch virtual interface (SVI)**

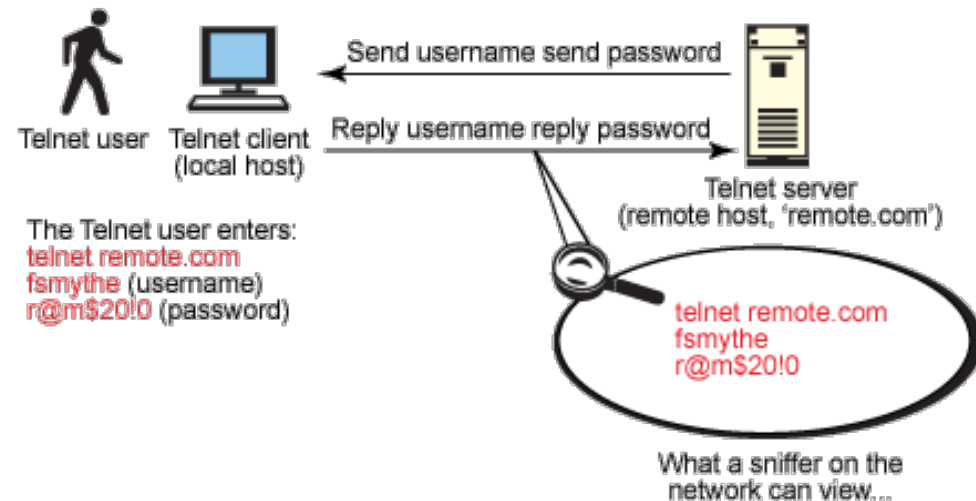
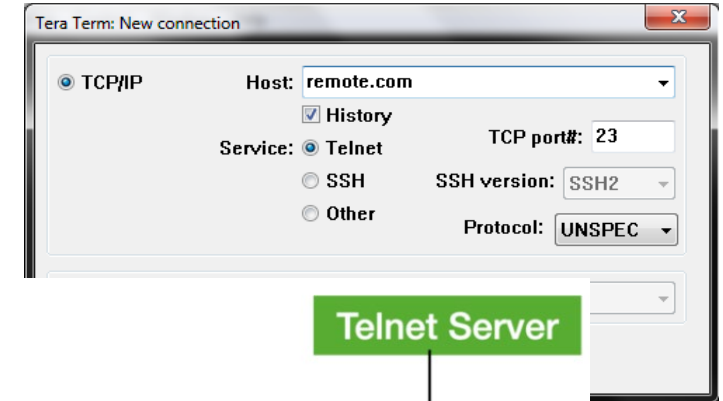
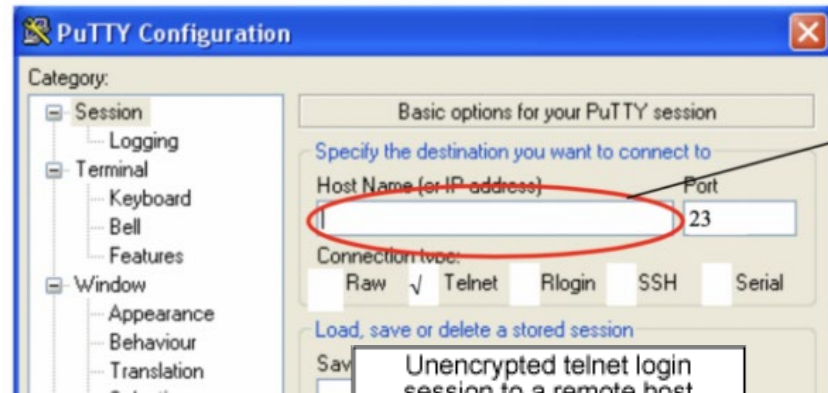


Configuring **SVI** using the command **interface vlan n**, the switch can now send and receive frames using the configured **IP address**.

```
switch(config)# interface vlan 1
switch(config-if)# ip address 192.168.1.200 255.255.255.0
switch(config-if)# no shutdown
switch(config-if)# exit
switch(config)# ip default-gateway 192.168.1.1
```

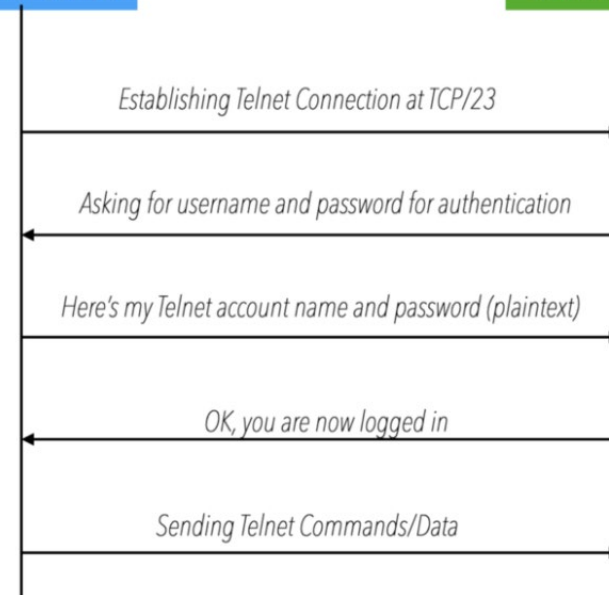
Note:
*interface vlan
and vlan are
different.*

With IP address, remote management can now be performed – typically using **Telnet** but is **not secure** because all messages are transmitted in the clear including login password!



Telnet Client

Telnet Server



Today, **SSH** (Secure Shell) is recommended so that all messages are protected within an established **secure channel** (you'll learn the theory in ICT2205 Applied Cryptography).

Configuring network device to support **SSH**:

```
switch(config)# hostname SW1
SW1(config)# ip domain-name remote.com
```

1. Configure domain name for key generation

```
SW1(config)# crypto key generate rsa modulus 2048
```

```
The name for the keys will be: SW1.remote.com
```

```
% The key modulus size is 2048 bits
```

```
% Generating 2048 bit RSA keys, keys will be non-exportable...
```

```
[OK] (elapsedtime was 74 seconds)
```

2. Generate a key-pair

```
SW1(config)# ip ssh version 2
```

3. Only support SSHv2 – recommended for security reason

```
SW1(config)# ip ssh timeout 120
```

4. Optional

```
SW1(config)# ip ssh authentication-retries 3
```

For troubleshooting, you may verify the status of **SSH** using the following commands:

```
SW1# show ip ssh
```

```
SSH Enabled - version 2.0
```

```
Authentication timeout: 120 secs; Authentication retries: 3
```

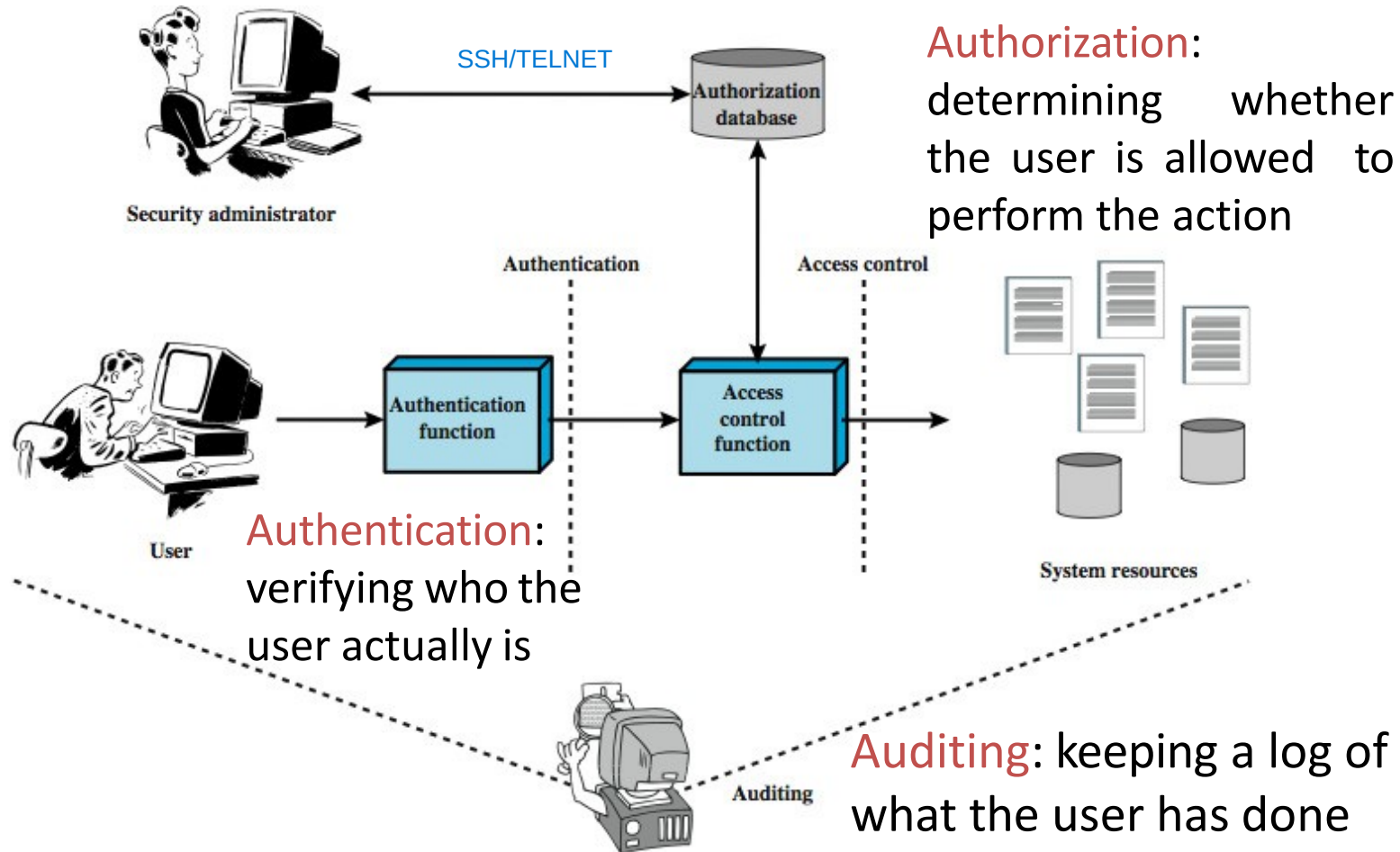
```
SW1# show ssh
```

Connection	Version	Mode	Encryption	Hmac	State	Username
0	2.0	IN	aes128-cbc	hmac-sha1	Session started	wendell
0	2.0	OUT	aes128-cbc	hmac-sha1	Session started	wendell

```
%No SSHv1 server connections running.
```

Above command shows that user Wendell is currently connected to switch SW1 via SSHv2.

After connecting to network devices, the next problem is how to manage **access control**, which is achieved by **authentication**, **authorization** and **accounting**, known as **AAA** (or triple-A).

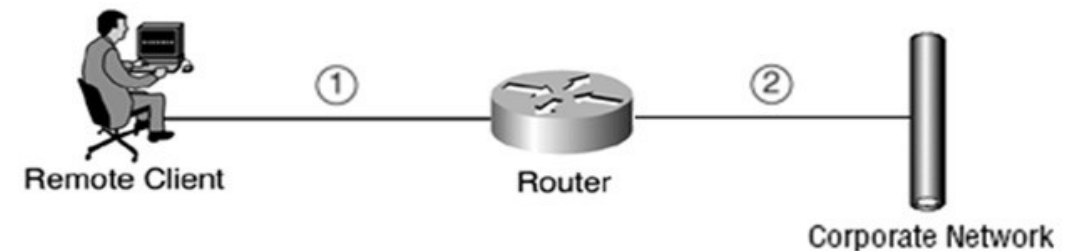


Broadly, there are two main uses of AAA for **access control** – **device administration** and **network access**. (Our focus will be on device administration.)

Device administration: controlling access to who can log in to a network device to configure and manage it.



Network access: essentially about verifying the identity of the user/ device before permitting to use the network.



Lab Exercise 1

8.1

Authentication is to verify who the user is and it can be implemented using:

- Locally stored usernames and passwords
- Centrally using **TACACS+** or **RADIUS** servers

Cisco supports **simple password security** – with **no username** – for console and telnet **login**. But everyone will use the same password!

```
switch(config)# line con 0
switch(config-line)# password hope
switch(config-line)# login
```



Console Password

```
switch(config)# enable password faith
```

User Mode

Enable Password

Enable Mode

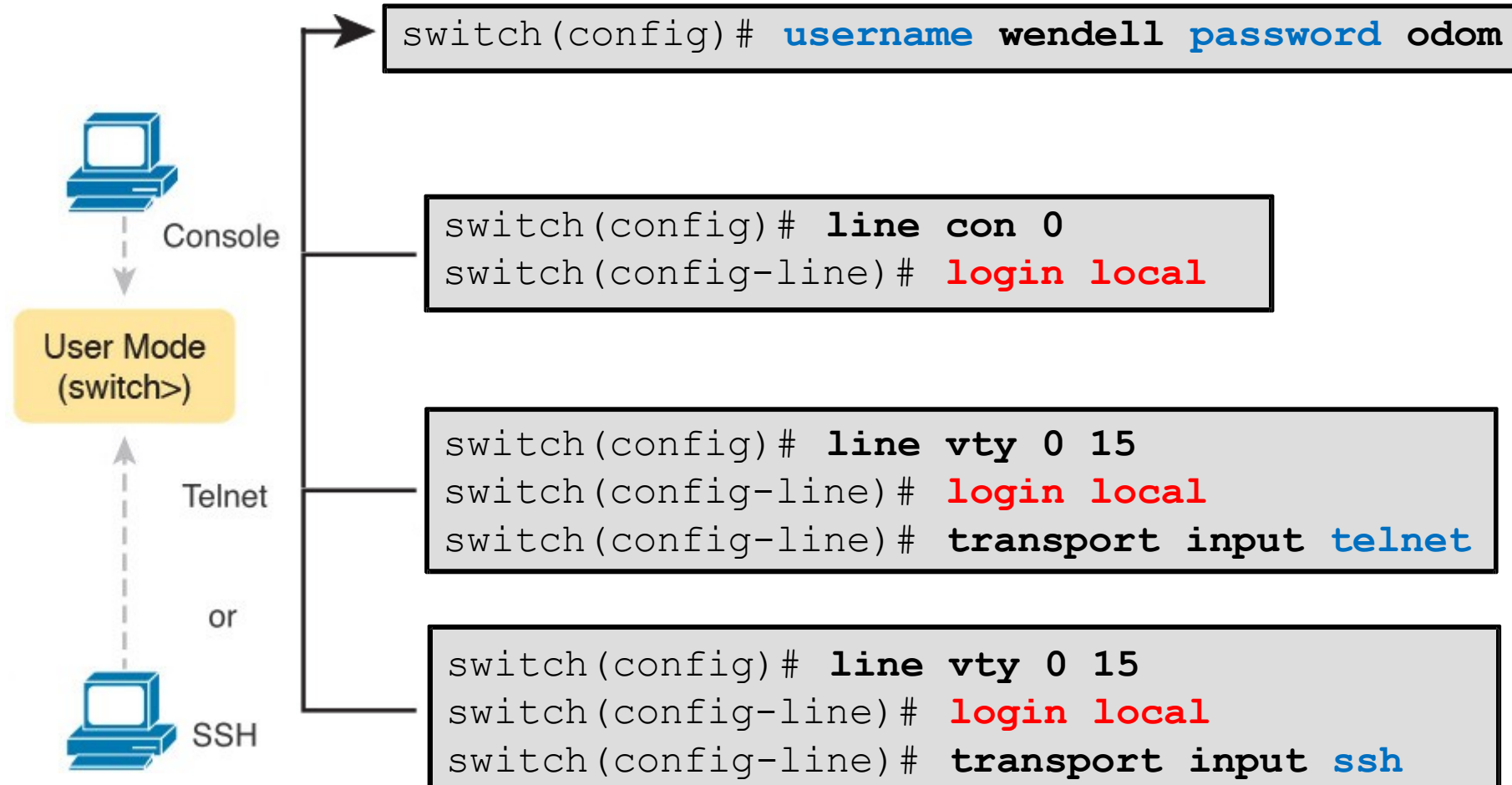


Telnet Password

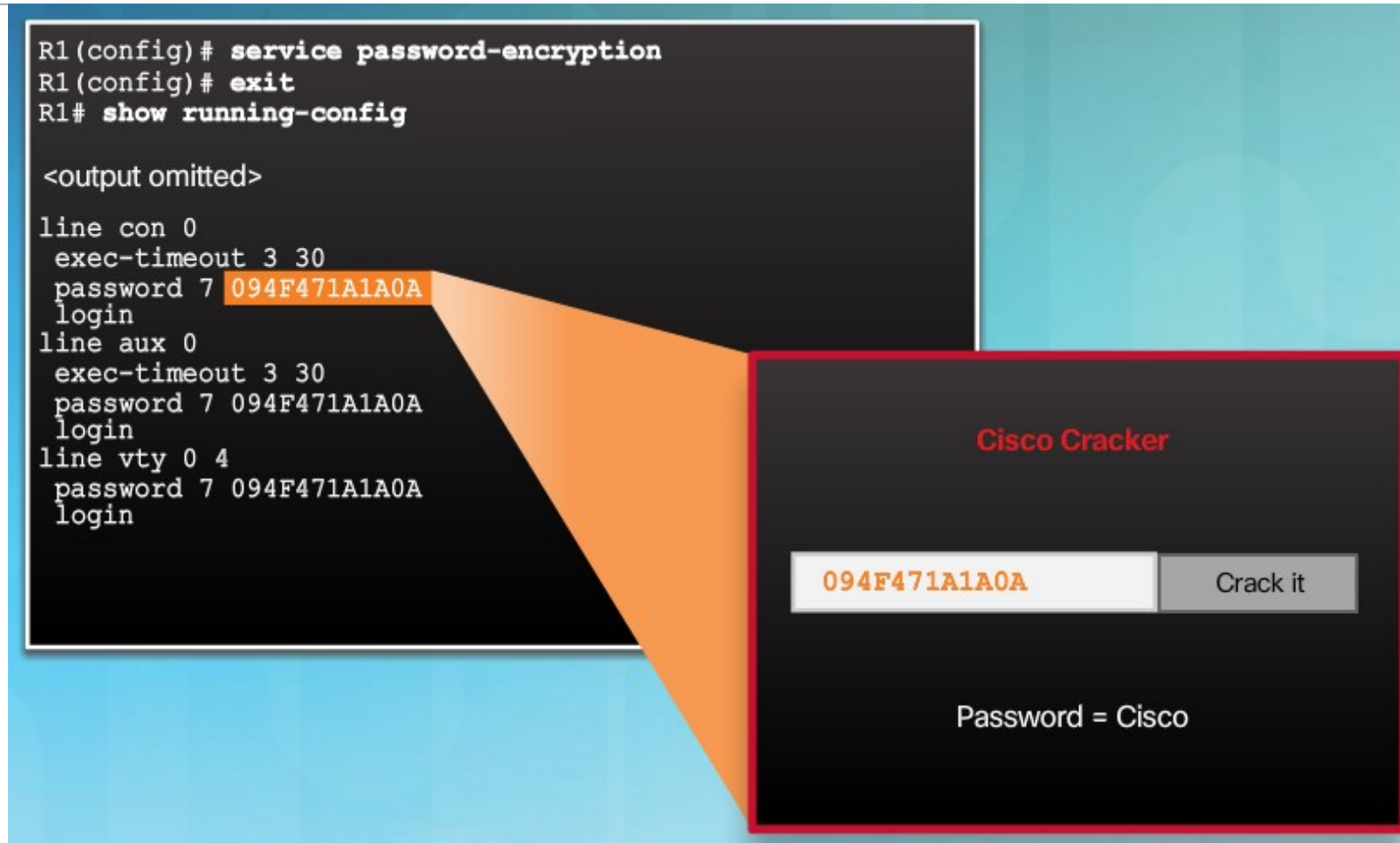
```
switch(config)# line vty 0 15
switch(config-line)# password love
switch(config-line)# login
```

- **password** *password-string* command defines the password
- **login** command enables simple password security
- **enable password** *password-string* command for enable mode password

A better **authentication** method will be to configure for each user a locally stored (**login local**) unique **username** and **password** which is a requirement for SSH.



By default, Cisco stores passwords in the configuration file in clear, which can be encrypted using the **service password-encryption** command but the protection is very weak



To provide better protection, Cisco has now implemented **cryptographic algorithms** like **md5**, **sha256** and **scrypt** which you will be studying the details in ICT2205 Applied Crypto.

By replacing '**password**' command with '**secret**', **MD5 hash** will be used to protect user passwords in the configuration file by default.

```
Router(config) #
```

```
username name secret unencrypted-password
```

```
Router(config) #
```

```
enable secret unencrypted-password
```

For even better protection, **sha256** and **scrypt** are recommended:

```
Router(config) #
```

```
username name algorithm-type {md5 | scrypt | sha256 } secret unencrypted-password
```

```
Router(config) #
```

```
enable algorithm-type {md5 | scrypt | sha256 } secret unencrypted-password
```

In addition, Cisco has added features to **slow down brute-force attack** on login as follows:

```
R1(config)#
```

```
login delay seconds
```

```
login block-for seconds attempts tries within seconds
```

To prevent the DoS attack from denying authorized administrators from login during this **block-for period** called **quiet-mode**, you may configure ACL to allow login before block-for timer expires as follows

```
R1(config)#
```

```
login quiet-mode access-class {acl-name|acl-number}
```

```
R1(config)# ip access-list standard PERMIT-ADMIN
R1(config-std-nacl)# remark Permit only Administrative hosts
R1(config-std-nacl)# permit 192.168.10.10
R1(config-std-nacl)# permit 192.168.11.10
R1(config-std-nacl)# exit
R1(config)# login quiet-mode access-class PERMIT-ADMIN
```

For **accounting** purpose (to be discussed later), it is also useful to **log login attempts**

```
R1(config)#
```

```
login on-success log [every login]
```

```
R1(config)#
```

```
login on-failure log [every login]
```

```
Router#  
*Nov 1 08:53:54.571: %SEC_LOGIN-4-LOGIN_FAILED: Login failed [user: wendell]  
[Source: 192.168.1.10] [localport: 23] [Reason: Login Authentication Failed -  
BadPassword] at 08:53:54 UTC Thu Nov 1 2018  
Router#
```

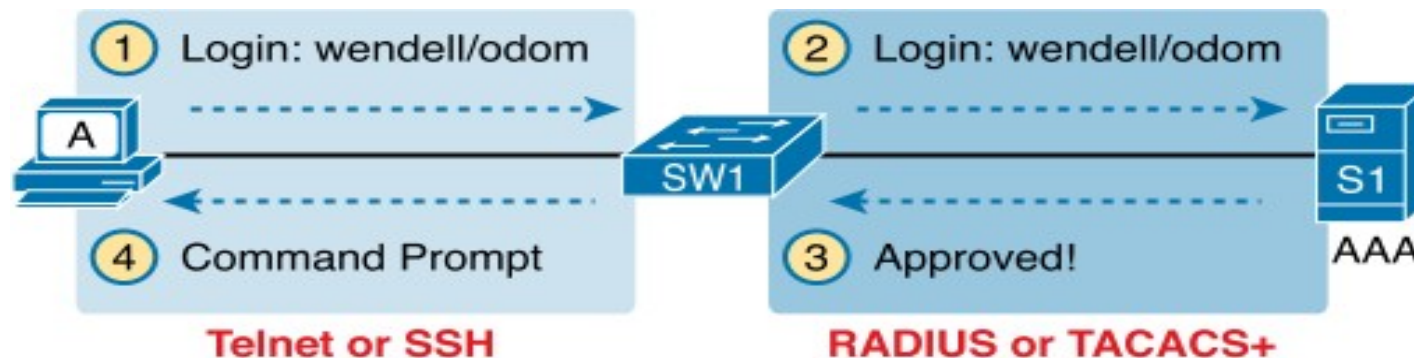
```
Router#  
Router#show login failures  
Total failed logins: 2  
Detailed information about last 50 failures
```

Username	lPort	Count	TimeStamp	SourceIPAddr
wendell	23	2	08:53:54 UTC Thu Nov 1 2018	192.168.1.10

A better and more practical way is to store passwords in an **external centralized AAA server (server-based AAA)** for authentication, especially when managing many devices.

Benefits of **server-based AAA**:

- **Scalability** and **manageability**: the configuration of usernames and passwords is centralized and not repeated in every device
- **Fault tolerance**: multiple AAA servers can be configured for backup purpose



Communication between **network devices** and **AAA servers** typically uses **RADIUS** or **TACACS+** (Terminal Access Controller Access-Control System, pronounced as 'tack-axe' from Cisco textbooks) protocols.

To enable authentication using centralized AAA server, configure using **AAA commands** with server on VRF, e.g. with **TACACS+** as follows:

```
SW1(config)# aaa new-model
```

1. Enable AAA

```
SW1(config)# aaa group server tacacs+ mTACACS
```

2. Specify AAA server

```
SW1(config-server-tacacs)# server-private 10.250.250.1 key  
cisco123
```

```
SW1(config-server-tacacs)# ip vrf forwarding oam
```

```
SW1(config-server-tacacs)# ip tacacs source-interface  
GigabitEthernet1
```

3. Configure authentication method list

```
SW1(config)# aaa authentication login MY-LIST group mTACACS
```

4. Apply authentication method list

```
SW1(config)# line vty 0 15
```

```
SW1(config-line)# login authentication MY-LIST
```

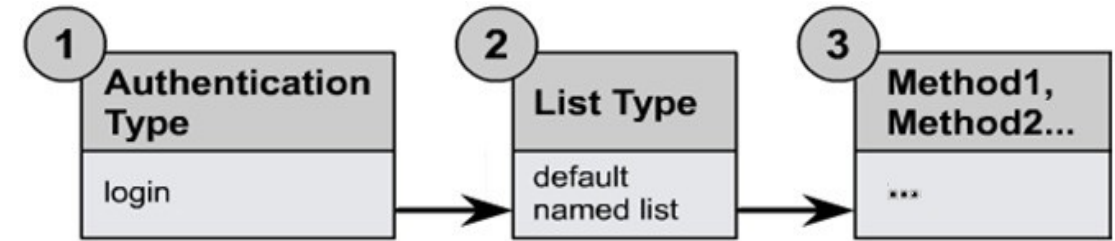
```
SW1# test aaa group mTACACS sbng cisco123 legacy
```

```
Attempting authentication test to server-group mTACACS using  
tacacs+
```

```
User was successfully authenticated.
```


Here is an explanation of the common **methods** for use with **aaa authentication** command to configure the **method list**:

```
Router(config)#aaa authentication type {default | list-name}  
method1[...method4]
```



Methods	Description
local	Use local username for authentication.
local-case	Use case-sensitive local username for authentication.
enable	Use enable password for authentication.
none	No authentication.
group radius	Use the configured RADIUS servers for authentication.
group tacacs+	Use the configured TACACS+ server for authentication.
group group-name	Use the configured RADIUS or TACACS+ servers for authentication as defined by the command aaa group server radius group-name or aaa group server tacacs+ group-name

AAA server setup (tac_plus)

Steps on setting up the opensource server on Kali-linux:

```
# apt install git make bison flex gcc libwrap0-dev
# cd /tmp
# git clone https://github.com/facebookarchive/tac_plus
# cd tac_plus/tacacs-F4.0.4.28/
# ./configure --libdir=/usr/lib64/
# make
# make install (root install)
# tac_plus -C /etc/tac_plus.config -d 32 -l /var/log/tac.log
```



```
key = cisco123
group = netadmin {
    default service = permit
    service = exec {
        priv-lvl = 15
    }
}
user = support {
    member = netadmin
    login = cleartext cisco123
}
user = $enab15$ {
    login = cleartext letmein
}
```

For **redundancy**, AAA can be configured with multiple methods, e.g. **TACACS+** server, **RADIUS** server, and **local database** (to avoid being locked out if AAA servers are down).

```
R1(config)# aaa new-model
R1(config)# tacacs server TACACSERVER
R1(config-server-tacacs)# address ipv4 192.168.1.11
R1(config-server-tacacs)# key TACACS-pa55w0rd
R1(config)#
R1(config)# radius server RADIUSERVER
R1(config-radius-server)# address ipv4 192.168.1.12 auth-port
1812 acct-port 1813
R1(config-radius-server)# key RADIUS-pa55w0rd
R1(config)#
R1(config)# username ADMIN secret Str0ngPa55w0rd
R1(config)#
R1(config)# aaa authentication login default group tacacs+
group radius local
```

An example of **RADIUS** server is the free open source and most widely deployed **freeRADIUS** server which you may also try out in your team project.

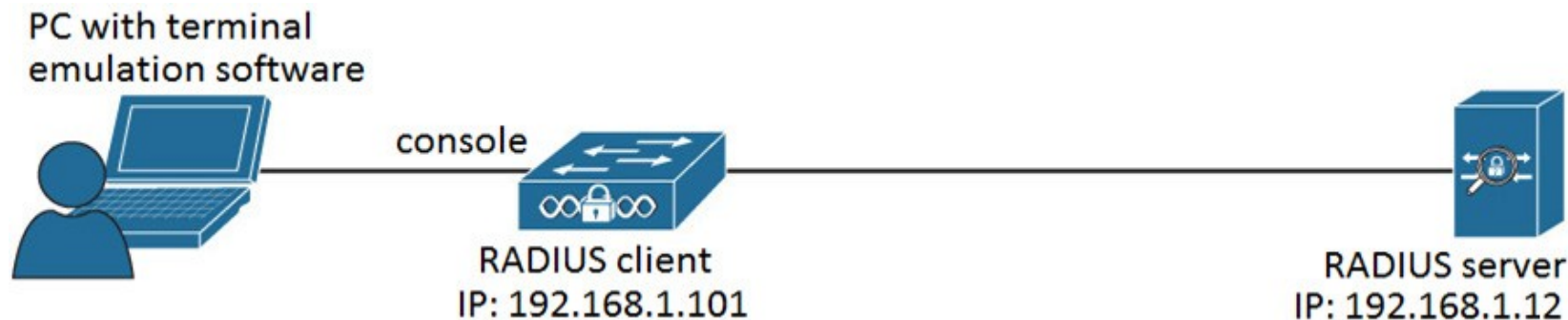
It's easier to install and run freeRADIUS in Kali Linux, follow the guide at [https://wiki.freeradius.org/guide/Getting Started](https://wiki.freeradius.org/guide/Getting%20Started)

In **clients.conf** specify IP addresses of supported network devices and shared password:

```
client networkdevices {  
    ipaddr      = 192.168.1.0/24  
    secret      = RADIUS-pa55w0rd  
}
```

In file "**users**", configure username and password:

```
wendell Cleartext-Password := "odom"
```

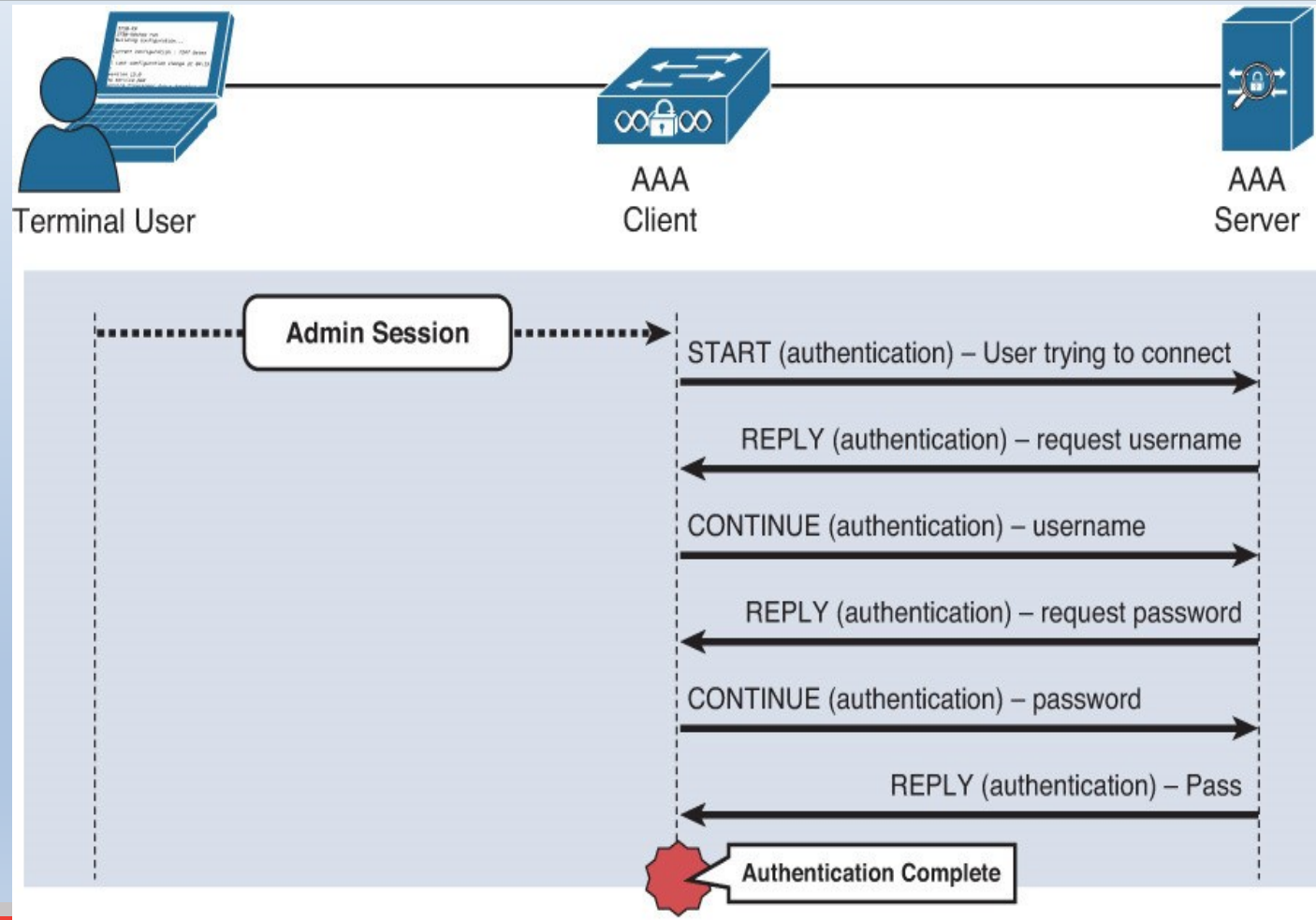


In summary, here is a table comparing **TACACS+** vs **RADIUS** for implementing **AAA**.

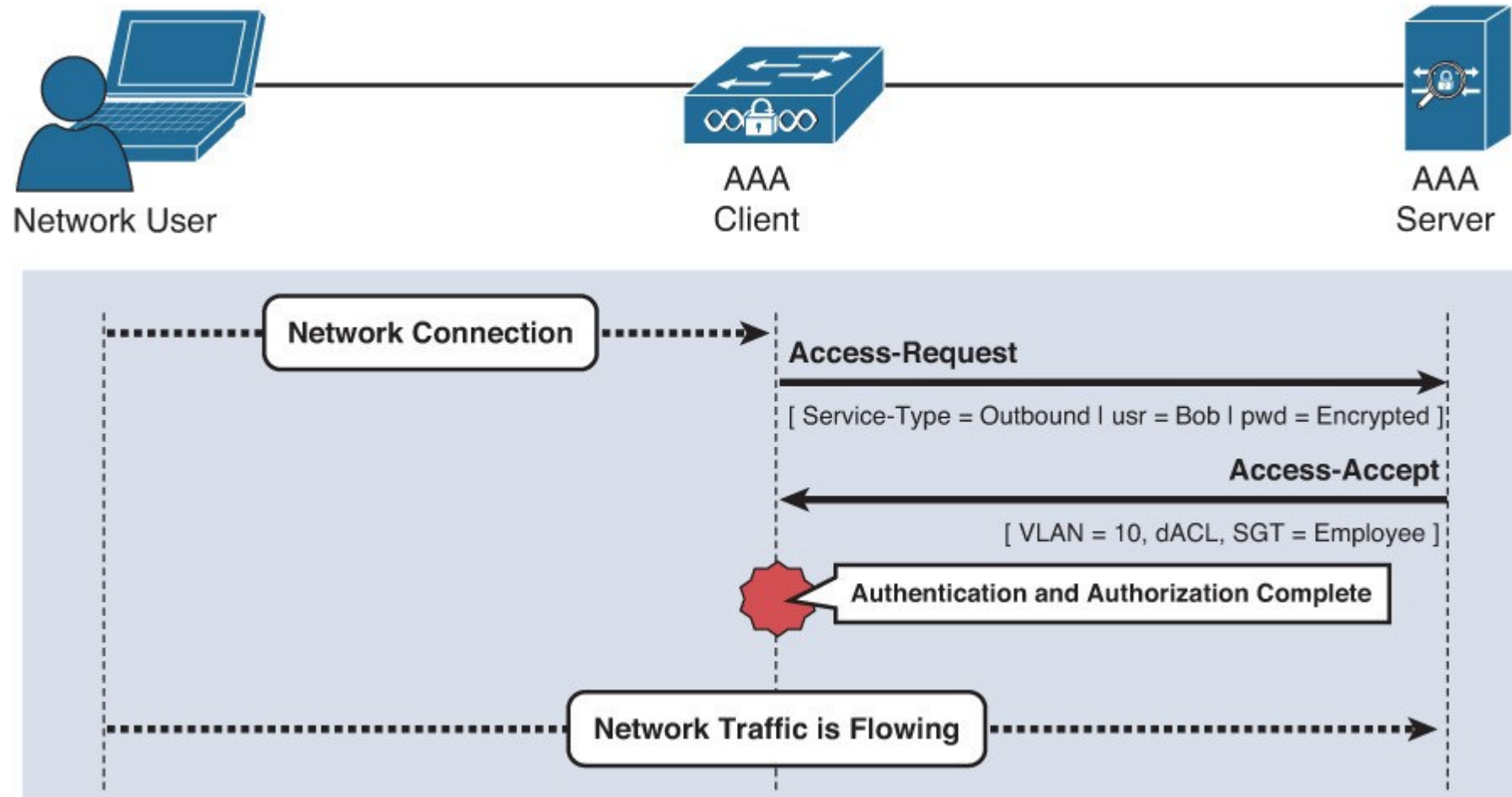
	TACACS+	RADIUS
Functionality	Separates authentication, authorization and accounting, allowing modularity of the security server implementation	Combines authentication and authorization but separates accounting, allowing less flexibility in implementation than TACACS+.
Standard	Mostly Cisco supported	Open/RFC standard
Transport Protocol	TCP port 49	UDP port 1812 and 1813
Security	Entire packet encrypted	Password encrypted
Authorization	Provides authorization of commands on a per-user or per-group basis.	Has no option to authorize router commands on a per-user or per-group basis
Primary Use	Device administration	Network access

You'll study theory of authentication protocols in ICT2205 Applied Crypto.

Specifically, **authentication** in TACACS+ is done separately from authorization and accounting as shown:



In contrast, **authentication** in **RADIUS** is done together with authorization in the same Access-Accept reply message as shown:



Lab Exercise 2

8.2

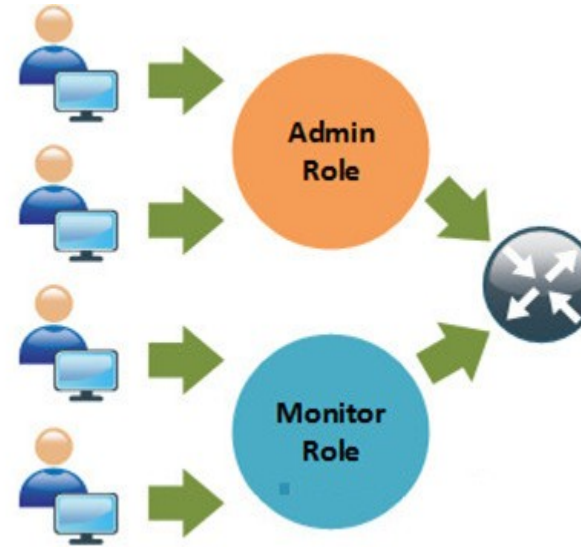
Authorization is to determine what the user is allowed to do and can be implemented using:

- Privilege level
- Role-based CLI views

After successful authentication, users should not be given more **access rights** than necessary which brings us to the topic of **authorization**.

Theoretically, there are different access control models for implementing **authorization**. The one widely used is the **role-based access control (RBAC)** model.

Assigning rights directly to each user



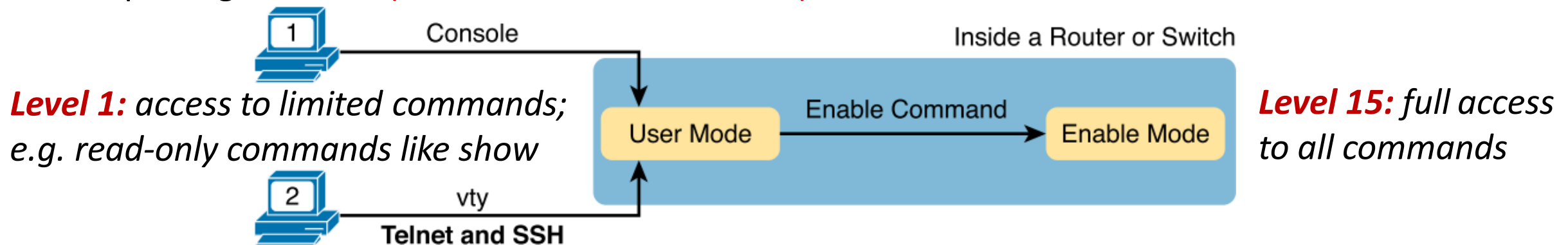
RBAC

Instead of assigning rights directly to users, **RBAC** assigns **rights to roles**, and **roles** can then be assigned **to users** according to job function.

By default, Cisco devices are implemented with **privilege levels** in 3 pre-defined levels to support **authorization**:

Privilege Level	Result
1	User level only (prompt is router>), the default level for login
15	Privileged level (prompt is router#), the level after going into enable mode
0	Seldom used, but includes five commands: disable, enable, exit, help, and logout

Upon login, users are placed at **level 1 (user mode)**, and can use **enable command** to move to privilege **level 15 (hence called enable mode)**.



Except for the five level 0 commands, all CLI commands are initially assigned to **level 1** or **level 15**, but can be reconfigured to different **custom privilege levels** between **2 – 14** as follows:

To reconfigure command(s) to new **privilege level**:

```
R1 (config) # privilege exec level level command-string
```

e.g.

```
R1 (config) # privilege exec level 5 ping
```

For security, a **password** can be configured for the new **privilege level**;

e.g.

```
R1 (config) # enable secret level 5 cisco5
```

To reset command(s) to default privilege level:

```
R1 (config) # privilege exec reset command-string
```

```
R1> show privilege
Current privilege level is 1
R1# ping 10.10.10.1
      ^
% Invalid input detected at '^' marker.

R1> enable 5
Password: <enter cisco5>
R1#
R1# show privilege
Current privilege level is 5
R1#
R1# ping 10.10.10.1

Type escape sequence to abort.
Sending 5, 100-byte ICMP Echos to 10.10.10.1, timeout is 2 seconds:
!!!!
Success rate is 100 percent (5/5), round-trip min/avg/max = 1/2/4 ms
R1#
```

To run a command, a user must be at the same or higher privilege level as the command as shown above

To simplify administration, **privilege levels** can be assigned to users directly so that they will enter at the assigned privilege level upon successful login.

An example of assigning 4 different users – USER, SUPPORT, JR-ADMIN and ADMIN – with different **privilege levels** to access the network device:

```
R1 (config) # username USER privilege 1 secret cisco
R1 (config) #
R1 (config) # privilege exec level 5 ping
R1 (config) # enable secret level 5 cisco5
R1 (config) # username SUPPORT privilege 5 secret cisco5
R1 (config) #
R1 (config) # privilege exec level 10 reload
R1 (config) # enable secret level 10 cisco10
R1 (config) # username JR-ADMIN privilege 10 secret cisco10
R1 (config) #
R1 (config) # username ADMIN privilege 15 secret cisco123
```

If using AAA, **privilege levels** can also be assigned directly to users using the **AAA authorization** commands as follows:

```
R1(config)# aaa new-model
R1(config)# ...
R1(config)# aaa authentication login default group tacacs+
group radius local
R1(config)# aaa authorization exec default group tacacs+
group radius local
```

Correspondingly, users in AAA servers must be configured with appropriate privilege levels, e.g.

- For tac_plus, in **tac_plus.config**

```
service = exec {
    priv-lvl = 15}
```

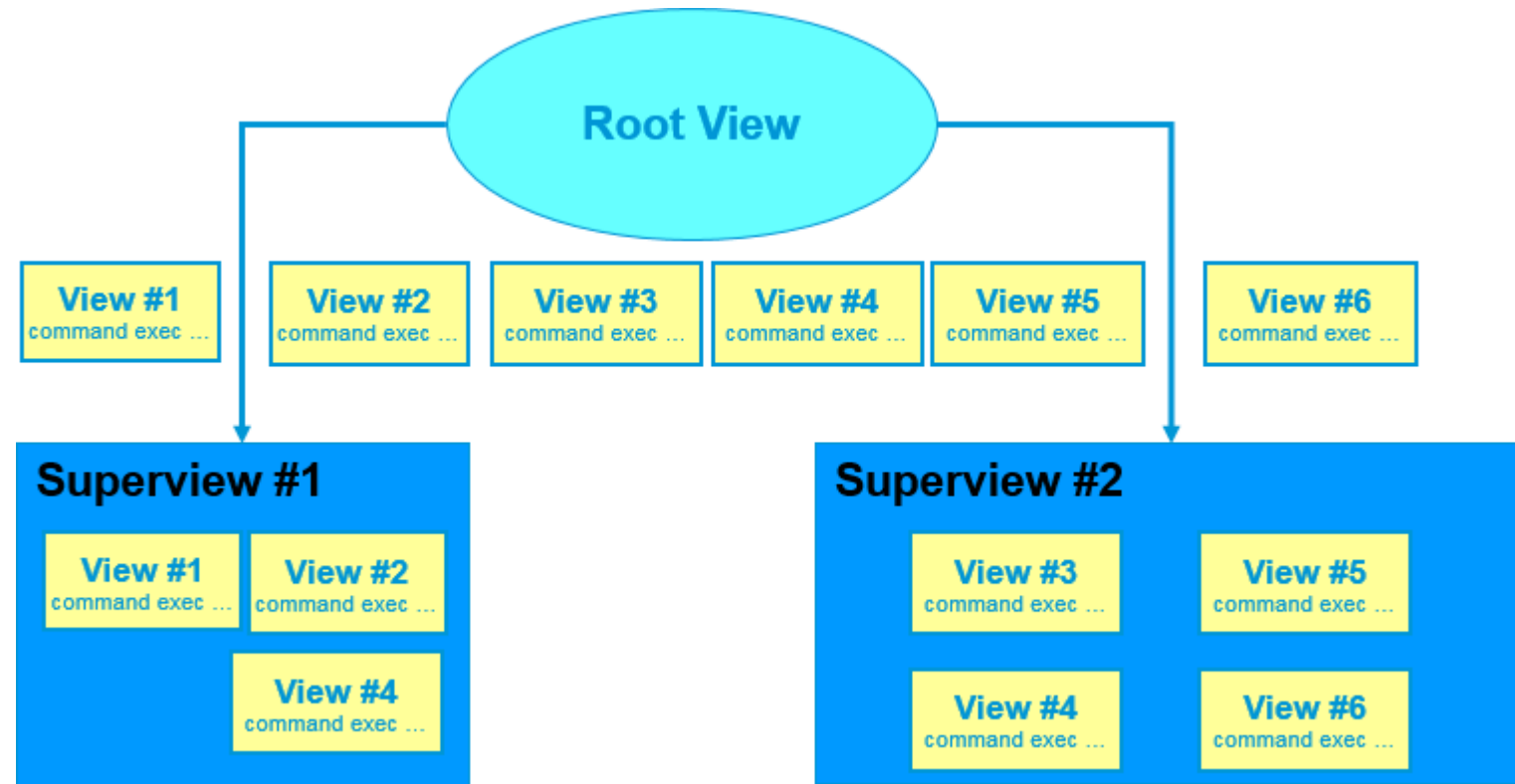
- For freeRADIUS, in file '**users**':

```
wendell Cleartext-Password := "odom"
cisco-avpair = "shell:priv-lvl=5"
```


However, privilege level has the problem that commands at lower level are always available for higher privilege users. Thus a more flexible **RBAC** using **views** to define **roles** is created

Views allow you to **pick and choose** which commands are available according to roles, and are divided into 3 types:

- **Root view:** for creating views and supervises, similar as privilege level 15
- **CLI view:** for defining a set of available commands
- **Superview:** for grouping CLI views together, but not commands



To begin defining roles using views, **enable root view** in **aaa new-model** and then configure new **CLI view** as follows:

```
R1 (config) # aaa new-model
```

1. Enable AAA to create views

```
R1 (config) # exit
```

```
R1 # enable view
```

2. Go to root view

```
Password:
```

3. Enter the same enable secret password

```
R1 #
```

```
R1 # configure terminal
```

```
R1 (config) # parser view VIEWNAME
```

4. Create new CLI view

```
R1 #
```

```
R1 (config-view) # secret viewpassword
```

5. Password for new view

```
R1 (config-view) # command exec include ...
```

```
R1 (config-view) # command exec exclude ...
```

```
R1 (config-view) # command exec include all ...
```

```
R1 (config-view) # ...
```

6. Pick and choose commands for view

```
R1 (config-view) # exit
```

```
R1 (config) #
```

Note: Only a maximum of 15 views can be created.

The parameters for you to pick and choose commands for **CLI views** are as follows:

```
command parser-mode {include | include-exclusive | exclude}  
[all] command
```

Parameter	Description
<i>parser-mode</i>	Specifies the mode in which the specified command exists (e.g. exec mode).
include	Adds a command or an interface to the view and allows the same command or interface to be added to an additional view.
include-exclusive	Adds a command or an interface to the view and excludes the same command or interface from being added to all other views.
exclude	Excludes a command or an interface from the view; that is, users cannot access a command or an interface.
all	(Optional) Specifies a “wildcard” that allows every command in a specified configuration mode that begins with the same keyword or every subinterface for a specified interface to be part of the view.
<i>command</i>	(Optional) Specifies a command that is added to the view.

Some examples of CLI views:

```
R1(config)# parser view SHOWVIEW
R1(config-view)# secret cisco
R1(config-view)# commands exec include show version
R1(config-view)# exit
R1#
R1(config)# parser view VERIFYVIEW
R1(config-view)# secret cisco5
R1(config-view)# commands exec include ping
R1(config-view)# exit
R1#
R1(config)# parser view REBOOTVIEW
R1(config-view)# secret cisco10
R1(config-view)# commands exec include reload
R1(config-view)# exit
```

After configuration, you may verify available commands in the **CLI view** as follows:

```
R1> enable view SHOWVIEW
Password:
R1#
R1# ?
Exec commands:
  enable      Turn on privileged commands
  exit        Exit from the EXEC
  show        Show running system information
R1#
R1# show ?
  parser      Display parser information
  version     System hardware and software status
```

By default, enable, exit and show parser commands are always available.

Optionally, **superview** can be created using similar commands as follows:

Example: creating
superviews
USER,
SUPPORT &
ADMIN

```
R1(config)# parser view USER superview
R1(config-view)# secret cisco
R1(config-view)# view SHOWVIEW
R1(config-view)# exit
R1#
R1(config)# parser view SUPPORT superview
R1(config-view)# secret cisco1
R1(config-view)# view SHOWVIEW
R1(config-view)# view VERIFYVIEW
R1(config-view)# exit
R1#
R1(config)# parser view ADMIN superview
R1(config-view)# secret cisco2
R1(config-view)# view SHOWVIEW
R1(config-view)# view VERIFYVIEW
R1(config-view)# view REBOOTVIEW
R1(config-view)# exit
```

You may also verify available commands in **superviews** similarly as follows:

```
R1# enable view USER
Password:
R1# ?
Exec commands:
  enable  Turn on privileged commands
  exit    Exit from the EXEC
  show    Show running system information
```

```
R1# enable view ADMIN
Password:
R1# ?
Exec commands:
  enable  Turn on privileged commands
  exit    Exit from the EXEC
  ping    Send echo messages
  reload  Halt and perform a cold restart
  show    Show running system information
```

In addition, you may use **parser view** command to verify created views as follows:

```
R1# enable view
Password:
R1#
R1# show parser view
Current view is 'root'
R1#
R1# show parser view all
Views/SuperViewsPresent in System:
SHOWVIEW
VERIFYVIEW
REBOOTVIEW
USER *
SUPPORT *
ADMIN*
----- (*) represent superview-----
R1#
```


Similar as privilege levels, **views** can be assigned to users directly so that they will enter at the **assigned views** upon successful login



Locally in
network
device

```
R1(config)# username Wendell view ADMIN secret odom
```

or centrally using AAA servers:

```
R1(config)# aaa authentication login default group tacacs+  
group radiuslocal  
R1(config)# aaa authorization exec default group tacacs+  
group radiuslocal
```

in tac_plus.net:

```
service = exec {  
    cli-view-name = ADMIN  
}
```

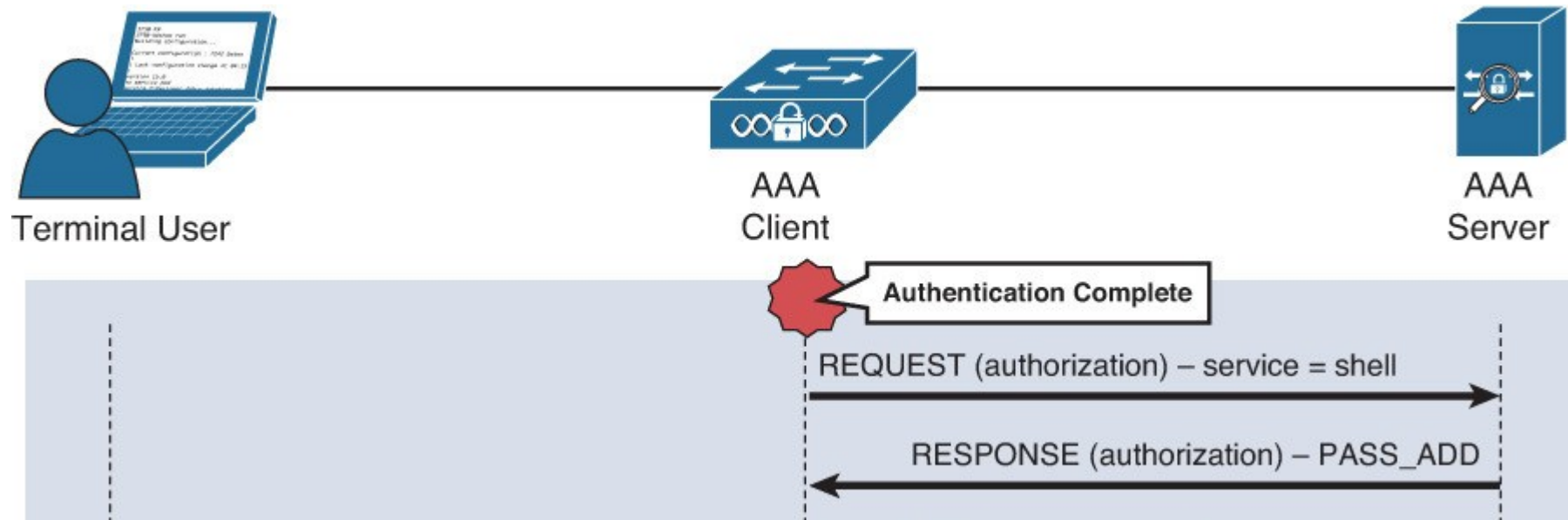
in freeRADIUS

'users':

```
Wendell Cleartext-Password := "odom"  
Service-Type = NAS-Prompt-User,  
cisco-avpair = "shell:cli-view-name=ADMIN"
```

For **TACACS+**, **authorization** is done separately from authentication as discussed earlier, whereas for RADIUS, authorization is done together with authentication.

Specifically, **TACACS+ authorization** consists of a request and a response message to assign privilege level or view as shown:



In contrast, **RADIUS authorization** is done together with authentication in the same access-request and access-accept message in slide 31.